



Pluviago Cultivation Pipeline

Complete Developer & Operations Reference

Everything a developer needs to understand the
Haematococcus pluvialis cultivation workflow — theory,
business logic, software implementation, validation gates, and
decision flows — from Flask to Packed batch.

Production Batch

Harvest Batch

Drying Batch

Contamination Incident

Return-to-Cultivation

Batch Split

Final Medium Batch

System: Pluviago — Frappe/ERPNext v15

Prepared: May 2026

Audience: Developers / Technical Team Leads

1 · Biological Background & Why It Matters

What Is Haematococcus pluvialis?

Haematococcus pluvialis is a freshwater green microalga renowned for producing **astaxanthin** — one of the most powerful natural antioxidants known, used in nutraceuticals, aquaculture feed, and cosmetics. Pluviago cultivates this organism at industrial scale inside photobioreactors (PBRs).

The organism has a unique two-phase life cycle that dictates the entire production strategy:

GREEN PHASE (VEGETATIVE GROWTH)

The alga grows rapidly under nutrient-rich, low-stress conditions. Cells are green, small, and actively dividing. This phase is purely about **biomass accumulation** — maximising cell count before the stress phase is induced.

- High nitrogen, high nutrients
- Optimal light and CO₂
- Target: maximum cell density
- No astaxanthin yet

RED PHASE (STRESS / ASTAXANTHIN ACCUMULATION)

Nutrient stress (nitrogen starvation), high light intensity, and/or high salinity trigger the alga to produce and accumulate astaxanthin as a protective pigment. Cells encyst and turn red-orange.

- Nitrogen-depleted medium
- High light (PAR) stress
- Target: maximum astaxanthin %
- Only at 6600L PBR stage

WHY THIS SHAPES THE SOFTWARE

The two-phase biology means the 6600L PBR batch is unique — it is the only stage that has a **phase transition** event (Green → Red). This is recorded in the software as a separate whitelist method with its own audit trail (date, operator, notes). No other stage has a phase concept.

The Cultivation Chain — Scale-Up Logic

Because microalgae cannot be inoculated directly into a 6600-litre reactor (contamination risk, insufficient density), cultivation follows a strict **sequential scale-up ladder**. Each stage multiplies the culture volume before it can be transferred to the next, larger vessel.



Each stage takes an **inoculum** (a portion of the previous culture) and adds fresh **final medium** to fill the new, larger reactor. The ratio of inoculum to fresh medium, the reactor volume, and the decision to scale up vs harvest are the core business decisions tracked in the software.

Stage	Reactor Volume	Generation	Phase	Key Decision
Flask	1 L	1	N/A	Scale Up → 25L PBR
25L PBR	25 L	2	N/A	Scale Up → 275L PBR
275L PBR	275 L	3	N/A	Scale Up → 925L PBR or Return-to-Flask
925L PBR	925 L	4	N/A	Scale Up → 6600L PBR
6600L PBR	6600 L	5	Green → Red	Harvest or Return-to-Flask
Harvest	—	—	—	QC → Drying or Reject
Drying	—	—	—	Yield → Packing

2 · Software Architecture Overview

Framework: Frappe / ERPNext v15

Pluviago is built as a **custom Frappe app** (module: `Pluviago Biotech`) sitting on top of ERPNext v15. All business logic lives exclusively in the Pluviago app — core Frappe/ERPNext files are never modified.

KEY FRAPPE CONCEPTS

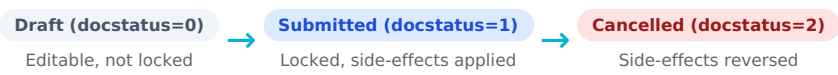
- **DocType** — the database model + form (like a class with a table and UI)
- **docstatus** — 0 = Draft, 1 = Submitted, 2 = Cancelled
- **Lifecycle hooks** — `validate`, `before_submit`, `on_submit`, `on_cancel`
- **before_submit** fires BEFORE docstatus is written to DB — throw here to cleanly block submission
- **on_submit** fires AFTER docstatus=1 is committed — never throw here (leaves inconsistent state)
- **Whitelist methods** — Python methods callable from the frontend via API (marked `@frappe.whitelist()`)
- **db_set** — directly updates a field in DB without triggering validate/on_submit hooks

The Seven Doctypes

DocType	Type	Submittable	Purpose
Pluviago Strain	Master	No	Strain registry — the organism being cultivated
Final Medium Batch	Transaction	Yes	Prepared growth medium consumed by PBs
Production Batch	Transaction	Yes	Core cultivation record — one per reactor stage
Production Batch QC	Child Table	—	QC readings embedded inside Production Batch
Contamination Incident	Transaction	Yes	Logs contamination events; flags the linked PB
Cultivation Return Event	Transaction	No	Audit record for Return-to-Cultivation withdrawals
Harvest Batch	Transaction	Yes	Records the harvest; triggers PB closure
Drying Batch	Transaction	Yes	Records drying run; writes yield back to Harvest Batch

Document Status Flow

All submittable doctypes follow the same status lifecycle:



⚠ CRITICAL IMPLEMENTATION RULE

In Frappe v15, `submit()` writes `docstatus=1` to the database *before* calling `on_submit()`. If `on_submit` throws, the document is left in an inconsistent state (docstatus=1 with no side-effects applied). **All submit-blocking validation must be in `before_submit()`**, which fires before the docstatus write and rolls back cleanly on exception.

3 · Production Batch — The Core Document

The **Production Batch** is the heart of the cultivation pipeline. Every reactor run — whether a 1-litre flask or a 6600-litre PBR — is one Production Batch document. Batches are linked parent-to-child to form the cultivation lineage tree.

Key Fields

Field	Purpose	Notes
current_stage	Which reactor size this batch is in	Flask / 25L PBR / 275L PBR / 925L PBR / 6600L PBR
generation_number	How many scale-ups from the original flask	1–5; enforced by validate; Return-to-Flask resets to 1
parent_batch	Link to the previous-stage PB that donated inoculum	Null for root Flask batches
strain	The Pluviago Strain being cultivated	Must be Active strain
stage_decision	What happens next: Scale Up / Harvest / Dispose	Must not be Pending at submission
status	Current business state of the batch	Auto-set by validate from stage_decision
reactor_volume	Capacity of the vessel in litres	Auto-set from stage (1 / 25 / 275 / 925 / 6600)
inoculum_volume_in	Culture volume taken from parent batch (L)	Deducted from parent's pool on submit
medium_volume_used	Final medium consumed from FMB (L)	Deducted from FMB.remaining_volume on submit
final_medium_batch	Which FMB the medium came from	Links to Final Medium Batch doctype
inoculum_volume_out	Total culture donated to children so far (L)	Auto-incremented when a child submits
culture_volume_available	Theoretical culture volume available for withdrawal	Set manually by operator
phase	Growth phase of 6600L batches only	Green Phase / Red Phase — one-way transition
contamination_status	Clean / Contaminated	Set by Contamination Incident creation
lineage_status	Active / Returned / Archived	Set by Return-to-Cultivation
qc_readings	Child table of QC records for this stage	At least one Pass reading required before Scale Up

Status State Machine

The `status` field is automatically derived from `stage_decision` in `validate()` :

stage_decision	status set to	Meaning
Scale Up	Scaled Up	Culture transferred to next stage; this batch is complete
Harvest	Harvested	Biomass ready for harvest; Harvest Batch can be created
Dispose	Disposed	Batch terminated (contamination, poor yield, etc.)
Pending	—	Submission blocked until decision is made

Stage Decision — The Operator's Core Choice

↑ Scale Up Culture is healthy. Transfer inoculum to the next larger PBR stage. Requires at least 1 QC reading with Pass result, no failed readings, no contamination detected, and batch not	Harvest Stop cultivation and harvest the biomass. Used when the 6600L has finished its Red Phase accumulation — or for early harvest of a contaminated or low-performing batch.	Dispose Terminate the batch without harvesting. Used when contamination renders the biomass unsafe, or yield is too low to justify processing. actual_completion is auto-set to today.
--	---	--

Submission Side-Effects (on_submit)

- **FMB deduction** — `medium_volume_used` is subtracted from the linked Final Medium Batch's `remaining_volume`
- **Inoculum deduction** — `inoculum_volume_in` is added to the parent batch's `inoculum_volume_out`

Cancellation Side-Effects (on_cancel)

- FMB volume is restored
- Inoculum is restored to parent's pool
- If no other submitted children remain, parent's status reverts to Active
- Batch status resets to Active

4 · Validation Gates & Business Rules

Every save, submit, or whitelist action runs through a series of validation gates. Understanding these is critical for both development and user support.

Production Batch — validate() Gates

Rule	Condition That Blocks	Error Message Trigger
Self-parent guard	<code>parent_batch == self.name</code>	"A batch cannot be its own parent"
Stage sequence	Child stage is not exactly one step above parent stage	"Stage sequence violation: from X the only valid next stage is Y"
Generation number — root	Root Flask must be Gen 1; root 25L must be Gen 2, etc.	"Root batch at Flask must have Generation Number = 1"
Generation number — child	Child gen $\neq$ parent gen + 1 (except Return-to-Flask, always Gen 1)	"Generation Number must be parent generation + 1"
Reactor capacity	<code>inoculum_in + medium_used > reactor_volume</code>	"Total volume exceeds reactor capacity"
Contamination gate	<code>contamination_status = Contaminated AND stage_decision = Scale Up</code>	"Cannot scale up a Contaminated batch"
Role gate	User lacks Production Supervisor/Manager/QA Head/Admin role for Scale Up	"Only a Production Supervisor or Production Manager can set Scale Up"
QC — no readings	<code>stage_decision = Scale Up AND qc_readings</code> is empty	"Cannot scale up: no QC readings recorded"
QC — failed reading	Any <code>qc_readings</code> row has <code>overall_result = Fail</code>	"Cannot scale up: N QC reading(s) have Overall Result = Fail"
QC — contamination flag	Any <code>qc_readings</code> row has <code>contamination_detected = 1</code>	"Cannot scale up: N QC reading(s) have Contamination Detected"

Production Batch — before_submit() Gate

Rule	Condition That Blocks
Pending decision	<code>stage_decision = "Pending"</code> — operator must commit to a decision before submitting

Harvest Batch — validate() Gates

Rule	Condition That Blocks
PB must exist	<code>production_batch</code> link is invalid
PB must be harvestable	<code>PB.status</code> not in [Active, Scaled Up, Contaminated, Harvested]
Stage decision check	<code>PB.status</code> is Active/Scaled Up AND <code>PB.stage_decision</code> $\neq$ Harvest
Volume cap	<code>harvested_volume > PB.reactor_volume</code>
Duplicate guard	Another submitted Harvest Batch already exists for the same PB

Harvest Batch — before_submit() Gate

Rule	Condition That Blocks
QC must be Passed	<code>qc_status</code> $\neq$ "Passed" — no submission until QC team approves

Drying Batch — validate() Gates

Rule	Condition That Blocks
HB must be submitted	HB.docstatus ≠ 1
HB status	HB.status not in [Approved, Packed]
Duplicate guard	Another submitted Drying Batch already exists for the same HB

Drying Batch — before_submit() Gate

Rule	Condition That Blocks
QC must be Passed	qc_status ≠ "Passed"

Contamination Incident — before_submit() Gates

Rule	Condition That Blocks
Decision must be set	decision = "Pending"
Status must be resolved	status not in [Resolved, Closed]
Root cause required	root_cause_category is blank — GMP traceability requirement

5 · Complete Operational Workflow — Step by Step

Phase 1: Pre-Cultivation Setup

1 Create / verify Pluviago Strain master

Navigate to Pluviago Strain → New. Enter Strain ID, Strain Name, Species, Status = Active. This is the master record that all Production Batches link to. A strain cannot list itself as its own parent strain.

2 Prepare and submit a Final Medium Batch

The growth medium (nutrients + water) is prepared in the medium lab before cultivation begins. A **Final Medium Batch (FMB)** is submitted once QC passes. The FMB tracks `remaining_volume` — each Production Batch deducts its `medium_volume_used` from this pool on submission. An FMB can serve multiple PBs until exhausted.

Phase 2: Flask Cultivation (Generation 1)

3 Create Root Flask Production Batch

Stage = Flask, Generation = 1, no parent_batch. The reactor_volume auto-sets to 1 L. Enter inoculation date, strain, and the FMB to use. Enter inoculum_volume_in (from stock culture) and medium_volume_used. The sum must not exceed 1 L (reactor capacity).

4 Add QC readings during cultivation

As the Flask batch grows, the QC team adds Process QC readings (pH, PAR, OD) and/or Biological QC readings (cell count, microscopy) to the `qc_readings` child table. At least one reading with Overall Result = Pass is required before Scale Up. Any contamination_detected = 1 or Overall Result = Fail will block Scale Up.

5 Set Stage Decision → Submit

When the Flask culture is ready, the Production Supervisor sets `stage_decision = Scale Up` and submits. On submit: (a) medium is deducted from FMB, (b) Flask status = Scaled Up. The Flask batch is now locked and the culture is ready to be split into the 25L PBR.

Phase 3: Scale-Up Chain (25L → 275L → 925L → 6600L)

6 Create child PB at each stage

For each scale-up: New Production Batch → set parent_batch = previous stage's PB name, current_stage = next stage (must follow Flask→25L→275L→925L→6600L exactly), generation_number = parent gen + 1. reactor_volume auto-fills. Enter inoculum_volume_in (culture taken from parent) and medium_volume_used from FMB.

7 Repeat QC → Decision → Submit at each stage

Each stage follows the same loop: add QC readings during cultivation, supervisor reviews results, sets stage_decision = Scale Up, submits. Each submission deducts inoculum from the parent batch's culture pool and medium from the FMB. The inoculum overshoot guard prevents taking more culture than what's available.

Phase 4: 6600L PBR — Green Phase

8 6600L PBR created in Green Phase

The 6600L batch begins as Green Phase. Operators add Process QC and Biological QC readings monitoring cell density, pH, PAR, OD600, and dry weight. Culture is growing vegetatively toward maximum biomass before stress induction.

9 Record Phase Transition: Green → Red

When the culture hits target density, the supervisor triggers **Record Phase Transition** (whitelist button on the submitted 6600L PBR form). Enters new_phase = Red Phase, transition_date, transitioned_by, and notes. The software records phase, phase_transition_date, phase_transitioned_by. This transition is one-way — Red Phase cannot revert to Green Phase.

Phase 5: 6600L PBR — Red Phase & Harvest Decision

10 Red Phase monitoring

Under nitrogen stress and high PAR, astaxanthin accumulates. Biological QC readings during Red Phase include

assay_value (% astaxanthin) and microscopy_result. Operators watch for encystment and colour change (green → orange-red).

11 Set Stage Decision = Harvest

When astaxanthin target is met, the supervisor updates stage_decision = Harvest on the submitted 6600L PBR (using db_set since it's already submitted). PB.status becomes Harvested, actual_completion = today. Now a Harvest Batch can be created for this PB.

5 · Complete Workflow (continued)

Phase 6: Harvest

12 Create Harvest Batch

New Harvest Batch → link production_batch (must be a submitted PB with stage_decision = Harvest). Enter harvest_date, harvested_by, harvested_volume (cannot exceed reactor capacity), wet_biomass_kg (wet weight of paste after centrifugation — the input to the Drying Batch). The QC team then fills qc_status, assay_type, assay_results, extract_purity, qc_checked_by, qc_date.

13 Submit Harvest Batch

`before_submit` checks qc_status = Passed. On submit: HB.status = Approved, PB.status = Harvested, PB.harvest_batch = HB name, PB.actual_completion = today. The Production Batch is now fully closed. Only one submitted HB per PB is allowed.

Phase 7: Drying

14 Create Drying Batch

New Drying Batch → link harvest_batch (must be submitted and status = Approved or Packed). wet_biomass_in auto-fills from HB.wet_biomass_kg. Enter drying method, temperature, drying start/end times. actual_dry_weight is the key output field. yield_percentage auto-calculates: $(\text{actual_dry_weight} / \text{wet_biomass_in}) \times 100$.

15 Submit Drying Batch

QC reviews the drying output, sets qc_status = Passed. `before_submit` checks this gate. On submit: DB.status = Approved. The following fields are written back to the Harvest Batch: HB.actual_dry_weight, HB.yield_percentage, HB.drying_batch. Only one submitted DB per HB is allowed.

Phase 8: Packing

16 Confirm Packing (whitelist method on HB)

After drying, the operator packs the dry powder. On the Harvest Batch form, fill: packing_date, packed_by, batch_size_packed, no_of_units, label_batch_no. Click **Confirm Packing** button. All three fields (packing_date, packed_by, label_batch_no) must be present or the method throws. On success: HB.status = Packed. The batch is now ready for dispatch.

Summary: Status at Each Stage

Document	After Submit	After Downstream Submit	After Cancel
Production Batch (Scale Up)	status = Scaled Up	—	status = Active; parent restored; FMB restored
Production Batch (Harvest)	status = Harvested	—	status = Active
Harvest Batch	status = Approved; PB.status = Harvested	—	status = Draft; PB.status = Active; PB.harvest_batch = None
Drying Batch	status = Approved; HB write-back applied	—	status = Draft; HB.dry_weight/yield/link cleared

6 · Special Flows

6.1 — Contamination Incident

When contamination is suspected during any cultivation stage, a **Contamination Incident (CI)** is raised. Creating a CI immediately flags the linked Production Batch.

AUTOMATIC SIDE EFFECTS — ON CI INSERT (AFTER_INSERT)

- PB.contamination_status → Contaminated
- PB.status → Contaminated
- Scale Up is now blocked for this PB (contamination gate in validate)

CI Lifecycle

1. Operator creates CI — fills incident_date, production_batch, contamination_type, status = Open
2. Investigation proceeds — decision field set (Harvest Immediately / Dispose / Continue with Monitoring)
3. When decision is set, decision_date and decision_by auto-populate in validate()
4. If batch_disposed = 1, disposal_date and disposal_by auto-set
5. Status updated to Resolved or Closed, root_cause_category filled
6. CI submitted — all three gates (decision, status, root_cause) checked in before_submit()

CI Cancellation Logic (on_cancel)

When a CI is cancelled, the system checks if *any other* open/submitted CIs still exist for the same PB. If yes → PB stays Contaminated. If no other CIs → PB restored: contamination_status = Clean, status = Active.

MULTI-CI SCENARIO

Two CIs can exist for the same PB (e.g., bacterial + fungal simultaneously). The PB remains Contaminated until ALL active CIs are cancelled. The last CI cancellation triggers the restore.

6.2 — Return-to-Cultivation

Return-to-Cultivation is a **material transfer** — a live culture withdrawal from a running 275L or 6600L batch to restart cultivation at Flask stage. The source batch is *not* closed; it continues operating. This preserves valuable strains and provides a fresh cultivation cycle from proven culture.

Eligibility Rules

- Source batch must be submitted (docstatus = 1)
- Source stage must be 275L PBR or 6600L PBR (not Flask/25L/925L)
- Source batch must not be Harvested or Disposed
- withdrawal_volume must be > 0
- withdrawal_volume must not exceed remaining culture (available – already_out)

What the Whitelist Method Does

1. Creates a new Flask Production Batch:
 - parent_batch = source batch name
 - current_stage = Flask
 - **generation_number = 1 always** (Roy's business requirement — each new Flask resets the generation count)
 - inoculum_volume_in = withdrawal_volume
 - medium_volume_used = dilution_volume (from specified FMB)
2. Creates a **Cultivation Return Event (CRE)** audit record:
 - source_batch, child_batch, withdrawal_volume, dilution_volume, return_date

- source_stage, source_phase, total_volume_to_flask (withdrawal + dilution)

3. Sets source batch lineage_status = Returned

⚠ GENERATION RESET RATIONALE

The Gen 1 reset on Return-to-Flask is a deliberate business rule. Each new Flask represents a fresh cultivation cycle starting from scratch, even if the inoculum came from a Gen 5 batch. The biological clock restarts at Gen 1 to track that Flask as the origin of a new cultivation run.

6 · Special Flows (continued)

6.3 — Batch Split

Batch Split allows one parent batch to spawn **multiple child batches simultaneously** at the next stage. This is used when a large culture is divided across multiple reactors (e.g., one 275L batch → two 925L reactors).

Rules

- n must be between 2 and 10 (at least 2 children, maximum 10)
- next_stage must be exactly one step above the parent's current_stage
- Parent must be submitted, not Harvested or Disposed
- If inoculum_volume_per_child is specified: total inoculum (n × per_child) must not exceed remaining culture

What It Creates

N Production Batch draft documents, each with: parent_batch = source, current_stage = next_stage, generation_number = parent_gen + 1, remarks = "Split X/N from [source]". Each child is submitted independently when ready.

6.4 — Final Medium Batch (FMB) Volume Tracking

Every Production Batch that uses medium deducts from its linked FMB's remaining_volume on submission. The FMB status auto-updates:

remaining_volume	FMB Status
= initial volume (nothing consumed)	Approved
0 < remaining < initial	Partially Used
= 0	Used

If a PB is cancelled, its medium_volume_used is restored to the FMB. Submitting a PB against a fully-exhausted FMB (remaining = 0) is blocked at on_submit.

6.5 — Inoculum Pool Tracking

Each parent batch maintains a culture pool: culture_volume_available (set by operator) minus inoculum_volume_out (auto-accumulated from children). When a child submits, its inoculum_volume_in is added to the parent's inoculum_volume_out. If the requested inoculum exceeds remaining culture, submission is blocked:

OVER-CONSUMPTION GUARD (ON_SUBMIT OF CHILD PB)

"Parent batch has only X L culture remaining. Requested inoculum: Y L."

When a child is cancelled, inoculum_volume_out is reduced back. If no other submitted children remain, parent status reverts to Active.

7 · Quality Control System

7.1 — Production Batch QC (Child Table)

Each Production Batch has a child table `qc_readings` where operators log QC results throughout the cultivation run. There are two types of QC readings:

PROCESS QC

- pH value
- PAR value (light)
- OD (Optical Density)
- Automation / PAR notes
- Culture phase (for 6600L)

Routine monitoring of process parameters. Done frequently throughout cultivation.

BIOLOGICAL QC

- Cell count (cells/mL)
- Cell size (µm)
- Dry weight (g/L)
- Assay value (% target compound)
- Microscopy result (Clean / Contaminated / Suspect)
- Contamination Detected flag

Gate Logic (for Scale Up decision)

- At least 1 QC reading must exist
- No reading may have `overall_result = Fail`
- No reading may have `contamination_detected = 1`
- Batch `contamination_status` must be Clean

7.2 — Harvest Batch QC

After the biomass is harvested and centrifuged, the QC team performs assay testing to verify quality before the batch can be submitted:

Field	Purpose
<code>qc_status</code>	Pending / Passed / Failed — must be Passed to submit HB
<code>assay_type</code>	HPLC / Spectrophotometry / Gravimetric / Other
<code>assay_results</code>	Free-text assay findings
<code>extract_purity (%)</code>	Purity of the extract
<code>qc_checked_by</code>	Link to User who performed QC
<code>qc_date</code>	Date of QC check

7.3 — Drying Batch QC

Drying batch QC confirms the dry powder meets specifications before the yield data is written back to the Harvest Batch:

Field	Purpose
<code>qc_status</code>	Passed / Pending / Failed — must be Passed to submit DB
<code>actual_dry_weight (kg)</code>	The key output — measured dry powder weight
<code>yield_percentage</code>	Auto-calculated: $(actual_dry_weight / wet_biomass_in) \times 100$
<code>moisture_content (%)</code>	Optional — measured moisture content of powder
<code>qc_checked_by</code>	Link to User who approved

YIELD CALCULATION — AUTO-COMPUTED IN `BEFORE_SAVE()`

$\text{yield_percentage} = (\text{actual_dry_weight} / \text{wet_biomass_in}) \times 100$

Calculated automatically whenever both values are present. This value is then written back to the Harvest Batch on Drying Batch submission.

8 · Data Flow & Cross-Document Side Effects

On Production Batch Submit

Action	Source Field	Target Doc / Field	Direction
Medium deduction	PB.medium_volume_used	FMB.remaining_volume – value	PB → FMB
Inoculum deduction	PB.inoculum_volume_in	parent_PB.inoculum_volume_out + value	PB → Parent PB

On Production Batch Cancel

Action	Effect
Restore medium	FMB.remaining_volume + medium_volume_used
Restore inoculum	parent_PB.inoculum_volume_out – inoculum_volume_in
Restore parent status	If no other submitted children → parent_PB.status = Active

On Harvest Batch Submit

Action	Source	Target
Status update	—	HB.status = Approved
PB closure	HB.name	PB.status = Harvested; PB.harvest_batch = HB.name; PB.actual_completion = today

On Harvest Batch Cancel

Action	Effect
HB reset	HB.status = Draft
PB restore	PB.status = Active; PB.harvest_batch = None

On Drying Batch Submit

Action	Source	Target
Status update	—	DB.status = Approved
Yield write-back	DB.actual_dry_weight	HB.actual_dry_weight
Yield write-back	DB.yield_percentage	HB.yield_percentage
Link write-back	DB.name	HB.drying_batch = DB.name

On Drying Batch Cancel

Action	Effect
DB reset	DB.status = Draft
HB clear	HB.actual_dry_weight = 0; HB.yield_percentage = 0; HB.drying_batch = None

On Contamination Incident Insert (after_insert)

Action	Effect
Flag PB	PB.contamination_status = Contaminated; PB.status = Contaminated

On Contamination Incident Cancel (on_cancel)

Condition	Effect
No other active CIs for the same PB	PB.contamination_status = Clean; PB.status = Active
Other CIs still active	PB remains Contaminated

9 · Developer Quick Reference

Stage Sequence Enforcement

STAGE_SEQUENCE CONSTANT IN PRODUCTION_BATCH.PY

```
[ "Flask", "25L PBR", "275L PBR", "925L PBR", "6600L PBR", "Harvested", "Disposed" ]
```

A child's stage index must equal parent's stage index + 1. Exception: Return-to-Cultivation Flask children always get stage = Flask regardless of parent's stage.

Reactor Volumes (Auto-Set)

Stage	reactor_volume	Expected Generation
Flask	1 L	Gen 1 (root) or Gen 1 (Return-to-Flask)
25L PBR	25 L	Gen 2
275L PBR	275 L	Gen 3
925L PBR	925 L	Gen 4
6600L PBR	6600 L	Gen 5

Whitelist Methods Summary

Method	DocType	What It Does	Guards
record_phase_transition()	Production Batch	Records Green→Red transition	Must be 6600L, submitted, not same phase, no reversal
create_return_batch()	Production Batch	Withdrawal + new Flask child + CRE	Submitted, eligible stage, not closed, volume > 0
create_split_batches()	Production Batch	Creates N child PBs at next stage	Submitted, n 2-10, stage sequence, not closed, inoculum within pool
mark_returned()	Production Batch	Sets lineage_status = Returned	Not Archived
archive_batch()	Production Batch	Sets lineage_status = Archived	—
confirm_packing()	Harvest Batch	Sets HB.status = Packed	status = Approved, packing_date, packed_by, label_batch_no all filled

Common Pitfalls for Developers

Pitfall	Detail	Fix
Throwing in on_submit	Frappe v15 commits docstatus=1 before on_submit runs. An exception in on_submit leaves the doc at docstatus=1 with no side-effects.	Always put submit-blocking checks in before_submit()
Saving submitted docs	Frappe blocks field edits on submitted docs unless field has allow_on_submit=1 in JSON	Use frappe.db.set_value() to bypass; or add allow_on_submit to the field definition
Cancelling draft docs	Only docstatus=1 docs can be cancelled. Cancelling a draft throws DocstatusTransitionError	Submit first, then cancel; or delete drafts

Reusing PBs in negative tests	A failed submit attempt (if check was in on_submit) leaves PB with docstatus=1, triggering duplicate guard on next HB creation	Use fresh PBs per negative test case; keep checks in before_submit
-------------------------------	--	--

Return-to-Flask gen number

Tempting to set `gen = parent_gen + 1`, but business requirement is always Gen 1

Hardcode `generation_number = 1` in `create_return_batch()`

Test Suite Location

E2E BACKEND TEST

```
apps/pluviago/pluviago/pluviago_biotech/tests/e2e_cultivation_test.py
```

Run with: `bench --site production.local execute pluviago.pluviago_biotech.tests.e2e_cultivation_test.run`

Coverage: 144 checks across 11 steps — strain setup, full scale-up chain, phase transition, harvest, drying, contamination incidents, return-to-cultivation, batch split, FMB exhaustion, inoculum tracking, negative/validation guards, confirm packing.